CHAPTER 06

RDS and Aurora

Aurora PostgreSQL · Connection Pooling · Backups · PITR · Performance Insights · CDK

AWS Tutorial · ShopFlow

06-00 Chapter Overview — RDS and Aurora

Amazon RDS (Relational Database Service) manages relational databases in the cloud. Instead of installing PostgreSQL, managing backups, handling failover, and patching the database engine yourself, RDS does all of that. Amazon Aurora is AWS's own database engine compatible with PostgreSQL and MySQL — it is 3x faster than standard PostgreSQL, stores data across 3 Availability Zones automatically, and costs 20% less than RDS PostgreSQL at high throughput. ShopFlow uses Aurora PostgreSQL 15 as the primary relational database for orders, users, and product catalog data.

■ RDS vs Aurora in Plain English

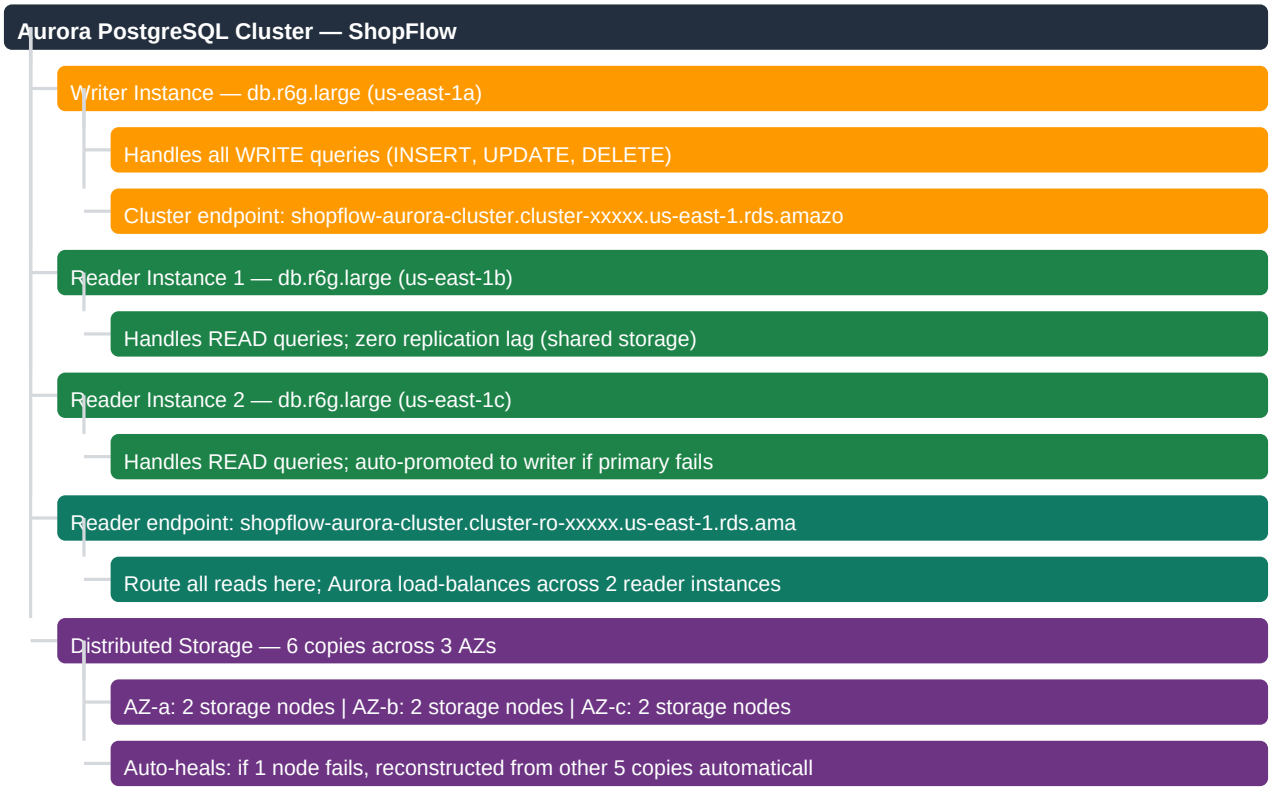
Standard RDS PostgreSQL: AWS manages the server, backups, and patches, but the database engine is standard PostgreSQL running on one or two servers. Aurora: AWS redesigned the storage layer — data is stored in a distributed cluster across 6 copies in 3 AZs. The writer sees consistent reads from all 15 read replicas instantly. Failover takes 30 seconds vs 90+ seconds for standard RDS.

Example: ShopFlow order-svc writes an order to Aurora. Aurora writes it to 6 storage nodes (2 per AZ). All 3 read replicas immediately see the new order because they share the same distributed storage. No replication lag for reads! standard PostgreSQL RDS: the replica gets the change via WAL replication after ~milliseconds delay.

Feature	Standard RDS PostgreSQL	Amazon Aurora PostgreSQL
Storage	Single EBS volume (replicated within one AZ if Multi-AZ)	Distributed across 6 copies in 3 AZs — auto-heals from disk fa
Read replicas	Up to 5; replication lag of ~100ms	Up to 15; zero replication lag (shared storage)
Failover time	90-120 seconds (standard Multi-AZ)	~30 seconds (Aurora keeps two warm replicas ready)
Throughput	Up to 80K IOPS (gp3 max)	Up to 120K IOPS; Aurora storage automatically scales to 128 T
Cost	~$0.48/hr for db.r6g.large	~$0.29/hr for db.r6g.large (40% cheaper at same performance
Backtrack	Not available	Rewind the database to any point in the last 72 hours without r

06-01 Aurora Architecture — How It Works

Aurora separates compute (the database server that processes queries) from storage (where data is actually stored). The storage layer is a distributed, fault-tolerant, self-healing cluster. 6 copies of your data are stored across 3 AZs. If one storage node fails, Aurora reconstructs it from the other 5 copies automatically — no human intervention, no downtime.



06-02 Creating Aurora PostgreSQL in the Console

Creating an Aurora cluster takes about 10 minutes in the console. The most important decisions: instance type (affects cost and performance), Multi-AZ (required for production — adds reader instances in other AZs), and encryption (always on for production databases).

■ Create ShopFlow Aurora PostgreSQL Cluster
1. Search "RDS" in the console → Databases → Create database
2. Creation method: Standard create
3. Engine type: Amazon Aurora
4. Edition: Amazon Aurora with PostgreSQL compatibility
5. Engine version: Aurora PostgreSQL 15.4 (latest LTS)
6. Template: Production
7. DB cluster identifier: shopflow-aurora-cluster
8. Credentials: Master username: shopflow_admin | Auto-generate password (copy after creation!)
9. Instance configuration: Burstable classes → db.t3.medium (dev/test) OR Memory optimized → db.r6g.large (production)
10. Availability and durability: Create an Aurora Replica in a different AZ (strongly recommended)
11. VPC: shopflow-vpc | Subnet group: create new: shopflow-aurora-subnet-group (data subnets)
12. VPC Security Group: shopflow-aurora-sg | Publicly accessible: No
13. Additional configuration: Initial database name: shopflow | Enable encryption: Yes
14. Backup retention: 7 days | Enable deletion protection: Yes
15. Click Create database — takes 5-8 minutes

■ aws › RDS › Create database › Aurora PostgreSQL

Create database — Connectivity

■ Connectivity

Virtual private cloud (VPC)

shopflow-vpc (vpc-0abc123) ■

■ DB subnet group: shopflow-aurora-subnet-group (covers data subnets in us-east-1a/b/c)

■ VPC security group (firewall)

VPC security group

■ Create new ■ Choose existing

Existing VPC security groups

shopflow-aurora-sg ■

■ Public access

Public access

■ Yes (DO NOT select this for production databases) ■ No (database is only accessible within the VPC)

■ Additional configuration

Initial database name

shopflow

■ Creates the "shopflow" database automatically on first boot

■ ■ Enable deletion protection (prevents accidental cluster deletion; remove before decommissioning)

Create database

06-03 Connecting to Aurora from Java — Connection Pooling

ShopFlow uses HikariCP (the default Spring Boot connection pool) to manage database connections efficiently. Connecting to a database is expensive — it takes ~100ms to establish a new TCP+TLS connection. Connection pools keep a set of open connections ready and reuse them across HTTP requests. ShopFlow configures HikariCP carefully: too few connections means request queuing; too many connections exhausts Aurora's max_connections limit.

```
# application.yml — Aurora connection with HikariCP
spring:
datasource:
# Use writer endpoint for writes; reader endpoint for reads
url: jdbc:postgresql://shopflow-aurora-cluster.cluster-xxxxx.us-east-1.rds.amazonaws.com:5432/s
hopflow
driver-class-name: org.postgresql.Driver
hikari:
# Max connections this service opens to Aurora
# Rule of thumb: (CPU cores × 2) + effective_spindle_count
# Aurora r6g.large has 2 vCPUs; effective formula: 2×2+1=5 per service
maximum-pool-size: 10 # 10 × 8 services = 80 total connections (Aurora handles ~1000)
minimum-idle: 3 # keep 3 warm connections even during quiet periods
connection-timeout: 3000 # fail fast if no connection available in 3s
idle-timeout: 600000 # close idle connections after 10 minutes
max-lifetime: 1800000 # replace connections after 30 minutes (avoid stale connections)
connection-test-query: SELECT 1 # verify connection is alive
```

```
# Read replica — route all read-only queries here
# Configure a second DataSource for reads:
datasource-read:
url: jdbc:postgresql://shopflow-aurora-cluster.cluster-ro-xxxxx.us-east-1.rds.amazonaws.com:543
2/shopflow
```

06-04 Aurora RDS Proxy — Connection Multiplexing for Serverless

When ShopFlow scales to 100+ ECS tasks, each with 10 connections, that is 1,000 database connections. Aurora can handle this, but Lambda functions are the bigger challenge: each Lambda invocation opens a new connection (Lambda cannot reuse connections across invocations). RDS Proxy sits between Lambda/ECS and Aurora: it maintains a pool of long-lived connections to Aurora, and hundreds of Lambda/ECS connections share the same small pool. ShopFlow uses RDS Proxy for all Lambda-to-Aurora access.

Scenario	Without RDS Proxy	With RDS Proxy
100 Lambda invocations simultaneously	100 new connections to Aurora; Aurora connection setup overhead, risk of hitting max_connections	100 Lambdas connect to RDS Proxy; Proxy maintains 10 Aurora
ECS task auto-scaling from 5 to 50 tasks	45 tasks × 10 connections = 450 new Aurora connections in 60 seconds; potential connection storm	450 new ECS connections to RDS Proxy; Proxy has existi
Aurora failover (writer goes down)	All 100 application connections get "connection reset"; apps must handle reconnection logic	RDS Proxy detects failover, holds connections open, redirects to new writer; appl

06-05 Aurora in CDK — Complete Cluster Setup

Here is the complete CDK code for the ShopFlow Aurora cluster — including the cluster, parameter group (database tuning), subnet group, security group, and automatic password rotation via Secrets Manager.

```
import * as rds from "aws-cdk-lib/aws-rds";
// Aurora PostgreSQL cluster
const auroraCluster = new rds.DatabaseCluster(this, "AuroraCluster", {
clusterIdentifier: "shopflow-aurora-cluster",
engine: rds.DatabaseClusterEngine.auroraPostgres({
version: rds.AuroraPostgresEngineVersion.VER_15_4,
}),
credentials: rds.Credentials.fromGeneratedSecret("shopflow_admin", {
secretName: "shopflow/aurora/master-credentials",
// Automatic rotation every 30 days via Secrets Manager
}),
defaultDatabaseName: "shopflow",
instances: 3, // 1 writer + 2 readers (one per AZ)
instanceProps: {
instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.LARGE),
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_ISOLATED},
vpc,
securityGroups: [auroraSg],
enablePerformanceInsights: true,
performanceInsightRetention: rds.PerformanceInsightRetention.DEFAULT_7_DAYS,
},
storageEncrypted: true,
deletionProtection: true,
backup: {retention: Duration.days(7), preferredWindow: "02:00-03:00"},
cloudwatchLogsExports: ["postgresql"],
cloudwatchLogsRetention: RetentionDays.ONE_MONTH,
});
```

06-06 Aurora Backups and Point-in-Time Recovery

Aurora provides two backup mechanisms: automated backups (retained for 1-35 days, free up to 100% of cluster storage size) and manual snapshots (retained until you delete them, charged at $0.021/GB/month). Point-in-Time Recovery (PITR) lets you restore the database to any second within the backup retention period — critical for

recovering from accidental data deletion.

■ Restore Aurora to a Point in Time
1. Scenario: a developer accidentally ran DELETE FROM orders WHERE status="pending" on production (should have been dev).
2. Go to RDS → Databases → shopflow-aurora-cluster → Actions → Restore to point in time
3. Restore to: Latest restorable time (or specify exact time: 2024-11-15 09:47:00 UTC — just before the accidental DELETE)
4. New DB cluster identifier: shopflow-aurora-cluster-pitr-recovery
5. Instance class: same as production (db.r6g.large)
6. VPC and subnet group: same as production
7. Click Restore DB cluster — takes 5-10 minutes
8. After restore: connect to the PITR cluster and verify data is correct
9. Extract missing data: INSERT INTO shopflow (production) SELECT * FROM shopflow_pitr WHERE status="pending"
10. Delete PITR cluster after data is recovered (costs ~$0.10/GB/day while running)

Backup Type	Retention	When To Use	Cost
Automated backup	1-35 days (configure in cluster settings)	PITR — restore to any second within retention window	Free up to 100% of DB storage size; $0.02
Manual snapshot	Indefinite (until you delete)	Pre-deployment checkpoint: "take a snapshot before this schema migration" long-term compliance	$0.021/GB/month for storage
Backtrack	Up to 72 hours (Aurora-specific)	Quickly rewind entire cluster without creating a new instance — faster than PITR	Enabled per cluster; $0.012/million change records

06-07 Aurora Performance Insights — Find Slow Queries

Performance Insights is a database monitoring feature that visualizes database load and identifies which SQL queries are causing performance problems. It shows: average active sessions (AAS), top SQL queries by load, top waits (what the database is waiting for — disk I/O, locks, CPU). ShopFlow uses Performance Insights to find the queries that cause order-svc latency spikes during peak traffic.

■ aws › RDS › shopflow-aurora-cluster › Performance Insights

Performance Insights — Last 1 hour

■ Database load (Average Active Sessions)

■ Current AAS: 3.2 | Max AAS: db.r6g.large has 2 vCPUs; if AAS > 2, database is overloaded

■ Top SQL (by contribution to database load — past 1 hour)

■ 48% SELECT o.*, u.email FROM orders o JOIN users u ON o.user_id = u.id WHERE o.status = $1 (avg: 380ms, calls: 12,450)

■ 31% UPDATE orders SET status = $1, updated_at = $2 WHERE order_id = $3 (avg: 12ms, calls: 8,200)

■ 12% INSERT INTO order_items (order_id, product_id, quantity, price) (avg: 8ms, calls: 18,000)

■ 9% SELECT * FROM products WHERE category_id = $1 ORDER BY rating DESC (avg: 450ms, calls: 2,100) ← MISSING INDEX

■ Top waits

■ io:DataFileRead: 67% of wait time — indicates insufficient buffer pool or missing index causing full table scans

View query details

■ The two slow queries (380ms orders JOIN and 450ms products by category) are caused by missing indexes. Add: CREATE INDEX CONCURRENTLY idx_orders_status ON orders(status); and CREATE INDEX CONCURRENTLY idx_products_category_rating ON products(category_id, rating DESC); — both will drop to < 5ms.

06-08 Database Schema Migration — Flyway with Spring Boot

ShopFlow uses Flyway for database migrations: version-controlled SQL scripts that evolve the schema safely. Each migration is a numbered SQL file. Flyway tracks which migrations have run in a schema_version table. On application startup, Spring Boot runs any pending migrations automatically. This ensures Dev, Staging, and Production databases are always in sync.

```
// resources/db/migration/V1__create_orders_table.sql
CREATE TABLE orders (
order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(user_id),
status VARCHAR(20) NOT NULL DEFAULT "pending"
CHECK (status IN ("pending","confirmed","shipped","delivered","cancelled")),
total_amount NUMERIC(10,2) NOT NULL,
currency VARCHAR(3) NOT NULL DEFAULT "USD",
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at DESC);
// V2__add_order_items_table.sql
CREATE TABLE order_items (
item_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
order_id UUID NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
product_id UUID NOT NULL,
quantity INT NOT NULL CHECK (quantity > 0),
unit_price NUMERIC(10,2) NOT NULL,
UNIQUE(order_id, product_id)
);
```

■ Always run migrations with CONCURRENTLY for index creation in production: CREATE INDEX CONCURRENTLY idx_name ON table(col). Without CONCURRENTLY, creating an index locks the table — blocking all INSERT/UPDATE during migration. CONCURRENTLY builds the index without a lock (takes longer but zero downtime).

06-09 Aurora Parameter Groups — Database Tuning

Aurora cluster parameter groups control the database engine configuration — equivalent to postgresql.conf. Default settings work for most workloads, but ShopFlow requires tuning for: 1M MAU workload, connection limits, logging, and memory allocation. Parameter group changes take effect without instance restart for most parameters.

Parameter	Default Value	ShopFlow Setting	Reasoning
max_connections	LEAST({DBInstanceClassMemory/9531392},5000)	1000	Limit connections per instance; use RDS Pro
shared_buffers	75% of RAM	75% of RAM (leave default)	Aurora manages buffer pool differently; chang
work_mem	4MB	16MB	Allows more complex sorts and hash joins in
log_min_duration_statement	-1 (disabled)	1000 (1 second)	Log all queries taking > 1 second; visible in C
log_lock_waits	off	on	Log when a query waits > deadlock_timeout f
random_page_cost	4.0	1.1	Tells planner that random reads are cheap (S
idle_in_transaction_session_timeout	0 (disabled)	60000 (60 seconds)	Kill transactions left open by poorly coded ap

06-10 Aurora Monitoring — CloudWatch and Enhanced Monitoring

Aurora publishes dozens of CloudWatch metrics. Enhanced Monitoring goes deeper — it runs an agent on the Aurora instance OS and publishes per-process CPU, memory, and I/O metrics at 1-second granularity. ShopFlow monitors: DatabaseConnections, CPUUtilization, FreeableMemory, CommitLatency, and ReadIOPS to detect problems before they affect customers.

■ aws › CloudWatch › Metrics › RDS › shopflow-aurora-cluster

Key Aurora Metrics to Monitor

■ Writer instance metrics

- ■ CPUUtilization: 34% (alert if > 70% sustained — scale up instance type)
- ■ DatabaseConnections: 82 (alert if > 800 for db.r6g.large — approaching max_connections)
- ■ CommitLatency: 2.1ms (write latency; alert if > 20ms — storage throughput issue)
- ■ FreeableMemory: 4.2 GB (alert if < 1 GB — memory pressure causing buffer eviction)

■ Storage metrics

- ■ VolumeBytesUsed: 47 GB (grows automatically; no action needed; Aurora auto-expands to 128 TB)
- ■ AuroraVolumeBytesLeftTotal: approaching 64 GB trigger (Aurora adds 10 GB increments automatically)

■ Replica lag (reader instances)

- ■ AuroraReplicaLag: 2ms (Aurora shared storage keeps this near-zero; alert if > 100ms — indicates storage issue)

Alert	Threshold	Action
CPUUtilization (writer)	> 70% for 5+ minutes	Investigate top queries in Performance Insights; scale up to db.r6g.xlarge
DatabaseConnections	> 80% of max_connections	Add RDS Proxy if not present; check for connection leaks in application c

Alert	Threshold	Action
CommitLatency	> 20ms	Check for lock contention in Performance Insights; may indicate heavy U
FreeableMemory	< 10% of instance RAM	Upgrade instance type; check for shared_buffers mis-configuration
ReplicationLag	> 100ms	Unusual for Aurora shared storage; investigate storage layer issues; con

06-11 Read-Write Splitting in Spring Boot

ShopFlow's read traffic is 10x its write traffic (customers browse 10 products for every 1 order placed). Routing all reads to Aurora reader instances and writes to the writer instance distributes load and allows the writer to focus on the most critical operations. Spring Boot supports read-write splitting with AbstractRoutingDataSource.

```
// Spring Boot: route reads to reader endpoint, writes to writer endpoint
@Configuration
public class DataSourceRoutingConfig {
@Bean("writerDataSource")
public DataSource writerDataSource() {
return DataSourceBuilder.create()
.url("jdbc:postgresql://shopflow-aurora-cluster.cluster-XXXX.us-east-1.rds.amazonaws.com:5432/shopflow")
.build();
}
@Bean("readerDataSource")
public DataSource readerDataSource() {
return DataSourceBuilder.create()
.url("jdbc:postgresql://shopflow-aurora-cluster.cluster-ro-XXXX.us-east-1.rds.amazonaws.com:5432/shopflow")
.build();
}
@Bean @Primary
public DataSource routingDataSource(
@Qualifier("writerDataSource") DataSource writer,
@Qualifier("readerDataSource") DataSource reader) {
AbstractRoutingDataSource router = new AbstractRoutingDataSource() {
@Override protected Object determineCurrentLookupKey() {
// Route to reader if current transaction is read-only
return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
? "reader" : "writer";
}
};
router.setTargetDataSources(Map.of("writer", writer, "reader", reader));
router.setDefaultTargetDataSource(writer);
return router;
}
}
// Service: @Transactional(readOnly=true) → uses reader endpoint
// @Transactional → uses writer endpoint
```

06-12 Aurora Serverless v2 — Dev and Test Environments

ShopFlow uses Aurora Serverless v2 for development and staging environments that are idle at night and on weekends. Serverless v2 scales to near-zero (0.5 ACUs) when idle and scales up instantly to handle load. Compared to a db.t3.medium ($0.068/hr) running 24/7, Serverless v2 for a dev database costs ~$2/day active + near-zero overnight — saving 60% per environment.

	Aurora Provisioned (db.r6g.large)	Aurora Serverless v2
Minimum cost	$0.29/hr × 730 hr = $212/month (runs even when idle)	0.5 ACU × $0.12/ACU-hr × ~400 active hrs = $24/month + $0

	Aurora Provisioned (db.r6g.large)	Aurora Serverless v2
Scale-up speed	Manual: change instance type (5-10 min downtime)	Automatic: 0.5 → 128 ACUs in seconds; no downtime
Predictability	Fixed performance (good for production SLOs)	Variable (scales based on load; occasional cold-start latency)
Best for	Production workloads with consistent traffic	Dev, test, staging; variable/unpredictable workloads; infreque

```
// CDK: Aurora Serverless v2 for staging/dev
const stagingDb = new rds.DatabaseCluster(this, "StagingDb", {
engine: rds.DatabaseClusterEngine.auroraPostgres({version:
rds.AuroraPostgresEngineVersion.VER_15_4}),
instances: 1, // single instance for dev (no HA needed)
instanceProps: {
instanceType: new ec2.InstanceType("serverless"),
vpc,
},
serverlessV2MinCapacity: 0.5, // near-zero when idle
serverlessV2MaxCapacity: 8, // scale up to 8 ACUs under load
});
```

06-13 Aurora Global Database — Multi-Region Setup

Aurora Global Database replicates data from a primary region to up to 5 secondary regions with < 1 second replication lag. ShopFlow plans to expand to Europe and Asia Pacific — Global Database means European customers read from a eu-west-1 read replica with < 10ms latency, while writes still go to the us-east-1 primary.

■ Create an Aurora Global Database

1. Go to RDS → Global databases → Create global database
2. OR: Select existing cluster → Actions → Add region
3. Primary cluster: shopflow-aurora-cluster (us-east-1)
4. Add secondary region: eu-west-1 (Ireland)
5. Instance class in secondary: db.r6g.large (match or smaller than primary)
6. VPC in secondary region: shopflow-eu-vpc (must be pre-created in eu-west-1)
7. Click Add region — takes 5-10 minutes to provision secondary cluster
8. Secondary cluster endpoint: shopflow-aurora-global.cluster-ro-eu-west-1.rds.amazonaws.com
9. European ECS tasks connect to the eu-west-1 reader endpoint for < 5ms latency

Global Database Feature	Description	ShopFlow Application
Replication lag	< 1 second (typically 100-200ms) from us-east-1 to eu-west-1	European customers reading product catalog see data written in t
Managed failover	Promotes secondary to primary in < 1 minute if primary region fails	If us-east-1 has a prolonged outage, promote eu-west-1 → all wri
RPO/RTO	RPO < 1 second; RTO < 1 minute	ShopFlow can achieve near-zero data loss and 1-minute recovery
Cross-region reads	Local read endpoints in each secondary region	ECS tasks in eu-west-1 read from local replica; zero cross-ocean

06-14 Database Indexing Strategy for ShopFlow

Indexes are the single most impactful database performance tool. A query that takes 2 seconds on a table with 10 million rows takes 2 milliseconds with the right index. ShopFlow's most common queries determine the indexing strategy. Every table needs: a primary key index (automatic), indexes on foreign keys (common JOIN columns), and indexes on WHERE clause columns used in high-frequency queries.

Table	Column(s) to Index	Query Pattern	Index Type
orders	user_id	SELECT * FROM orders WHERE user_id = ?	B-tree (default)
orders	status, created_at DESC	SELECT * FROM orders WHERE status="pending" ORDER BY created_at DESC	Composite B-tree (status, created_at DE
order_items	order_id	SELECT * FROM order_items WHERE order_id = ?	B-tree (foreign key)
products	category_id, rating DESC	SELECT * FROM products WHERE category_id=? ORDER BY rating DESC LIMIT 20	Composite B-tree (category_id, rating DE
products	name (full-text search)	Full-text product search by name	Use OpenSearch for full-text; not Postgre
users	email	SELECT * FROM users WHERE email = ? (login)	Unique B-tree (also enforces uniqueness
users	cognito_sub	SELECT * FROM users WHERE cognito_sub = ?	B-tree (Cognito user mapping)

```
-- Create all ShopFlow indexes (run as Flyway migration)
CREATE INDEX CONCURRENTLY idx_orders_user_id ON orders(user_id);
CREATE INDEX CONCURRENTLY idx_orders_status_ts ON orders(status, created_at DESC);
CREATE INDEX CONCURRENTLY idx_items_order_id ON order_items(order_id);
CREATE INDEX CONCURRENTLY idx_products_cat_rtg ON products(category_id, rating DESC);
CREATE UNIQUE INDEX CONCURRENTLY idx_users_email ON users(email);
CREATE INDEX CONCURRENTLY idx_users_cognito_sub ON users(cognito_sub);
```

06-15 Hands-On Lab — Set Up Aurora and Connect Spring Boot

This lab provisions a dev Aurora cluster, connects to it from Spring Boot, runs Flyway migrations, and verifies with a test API call. Estimated time: 20 minutes.

Step	Action	Verification
1	Create Aurora Serverless v2 dev cluster (console or CDK as shown in 06-12)	RDS → Databases → shopflow-dev → Status: Available
2	Note the cluster endpoint and copy the password from Secrets Manager	aws secretsmanager get-secret-value --secret-id shopflow/aurora/dev-cred
3	Add application-dev.yml with Aurora dev endpoint and HikariCP settings	File created; spring.datasource.url points to dev endpoint
4	Add Flyway dependency to pom.xml: org.flywaydb:flyway-core:10.0.0	mvn dependency:resolve shows flyway-core present
5	Create src/main/resources/db/migration/V1__create_tables.sql with the schema from 06-08	File created with orders and order_items tables
6	Run Spring Boot: mvn spring-boot:run	Console shows: Flyway: Successfully applied 1 migration; Tomcat started
7	Test: POST /api/v1/orders with a test order JSON body	HTTP 201 response; order_id UUID returned
8	Verify in Aurora: psql -h <endpoint> -U shopflow_admin -d shopflow -c "SELECT * FROM orders;	Inserted test order appears in the results
9	Enable Performance Insights: RDS → shopflow-dev → Modify → Enable Performance Insights	Before: Insights tab shows no data; After: Insights tab shows database load graph

06-16 Aurora Backtrack — Instant Time Travel

Aurora Backtrack is unique to Aurora — it can rewind the entire database cluster to any point within the past 72 hours in minutes, without creating a new instance. Unlike PITR (which creates a new cluster that takes 5-10 minutes), Backtrack rewinds in-place in seconds. Critical for: accidentally dropped a table, wrong WHERE clause in a batch UPDATE, test data leaked into production.

■ Use Aurora Backtrack to Recover from Accidental DELETE

1. Scenario: order-svc bug ran DELETE FROM order_items WHERE status IS NULL on production, deleting 50,000 rows
2. Go to RDS → Databases → shopflow-aurora-cluster → Actions → Backtrack
3. Current time: 2024-11-15 14:30:00 UTC
4. Backtrack to: 2024-11-15 14:22:00 UTC (just before the bad DELETE ran at 14:23)
5. Click Backtrack DB cluster — read the warning: ALL connections will be dropped
6. Confirm — Aurora rewinds storage to 14:22 UTC (takes 30-90 seconds)
7. Reconnect application — all 50,000 rows restored
8. Verify: SELECT COUNT(*) FROM order_items — should be back to original count
9. Note: Backtrack also rewinds any writes made between 14:22 and 14:30 — verify those were all part of the bad batch and not legitimate orders

Backtrack vs PITR	Backtrack	Point-in-Time Recovery (PITR)
Speed	30-90 seconds	5-15 minutes to create new cluster
How it works	Rewinds existing cluster in-place; no new resources	Creates a brand-new Aurora cluster from backup + WAL logs
Data continuity	Rewinds ALL data to that point; writes after backtrack time are lost	New cluster has data up to recovery point; original cluster un
Cost	$0.012/GB-hour of backtrack storage; 72-hour max window	No additional cost; uses existing automated backup storage
Best for	Quick recovery from accidents detected immediately	Recovery to older points; need original cluster to stay running

06-17 Partitioning Large Tables — Orders at Scale

At 1M MAU × 2 orders/month = 2M orders/month, the orders table grows to 24M rows/year. Queries without indexes become slow; even indexed queries scan many blocks. PostgreSQL declarative table partitioning splits the table into smaller partitions (one per month) that are queried independently. ShopFlow partitions the orders table by created_at month — queries for "show orders from November" only scan the November partition.

```
-- Partitioned orders table (range partition by month)
CREATE TABLE orders (
order_id UUID NOT NULL,
user_id UUID NOT NULL,
status VARCHAR(20) NOT NULL,
total_amount NUMERIC(10,2) NOT NULL,
created_at TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);
-- Create partitions (automate via pg_partman extension on Aurora)
CREATE TABLE orders_2024_11 PARTITION OF orders
FOR VALUES FROM ("2024-11-01") TO ("2024-12-01");
CREATE TABLE orders_2024_12 PARTITION OF orders
FOR VALUES FROM ("2024-12-01") TO ("2025-01-01");
-- Indexes on partitioned table apply to all partitions automatically
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status, created_at DESC);
-- Drop old partitions for data retention (replaces DELETE of millions of rows):
DROP TABLE orders_2017_01; -- instant; no table lock; frees storage immediately
```

06-18 Aurora Cost Optimization

Aurora is one of the largest line items in the ShopFlow AWS bill. Understanding the cost structure and optimization strategies can reduce it by 30-50% without any performance impact.

Cost Component	Price	ShopFlow Estimate	Optimization
Aurora writer instance (db.r6g.large)	$0.29/hr	$212/month	1-year Reserved: $0.198/hr = $145/month (32% saving
Aurora reader instances × 2	$0.29/hr × 2	$424/month	1-year Reserved: $0.198/hr × 2 = $289/month
Aurora storage ($0.10/GB-month)	$0.10/GB	$5/month (50 GB)	Storage auto-scales; compress with pg_compress for o
Aurora I/O ($0.20/million requests)	$0.20/M	~$20/month (100M I/O)	Aurora I/O Optimized cluster type: pay 25% more for st
Backtrack storage ($0.012/GB-hr)	$0.012/GB-hr	~$8/month (72hr window × 50GB)	Reduce backtrack window to 24 hours if sufficient; save
Data transfer (out to ECS)	$0.02/GB	~$5/month (250 GB queries)	Keep ECS and Aurora in same AZ to avoid cross-AZ ch

■ Aurora I/O Optimized — When to Switch

Aurora I/O Optimized costs 25% more for storage but has zero I/O charges. At ShopFlow's current scale (~100M I/O/month = $20/month in I/O charges), the standard pricing is better. Switch to I/O Optimized when I/O charges exceed the 25% storage premium. Rule of thumb: if I/O charges > 25% of your storage cost, switch. The console shows both costs — compare monthly.

06-19 Aurora Zero-ETL Integration with Redshift

Aurora Zero-ETL is a newer AWS feature that automatically replicates data from Aurora PostgreSQL to Amazon Redshift in near real-time — without writing ETL pipelines. ShopFlow uses this for analytics: the orders and products tables in Aurora are automatically available in Redshift for BI queries, without affecting Aurora performance.

■ Zero-ETL vs Traditional ETL

Traditional ETL: write a Glue/Lambda job that reads from Aurora, transforms data, writes to Redshift. Runs every hour. Up to 1-hour lag. Needs maintenance when schema changes. Zero-ETL: configure once in the console. Aurora automatically replicates changes to Redshift within seconds. Schema changes propagate automatically. No code to maintain.

Example: ShopFlow analytics team wants to run: SELECT product_id, SUM(quantity) FROM order_items GROUP BY product_id ORDER BY sum DESC. With Zero-ETL, this query runs in Redshift against live Aurora data (seconds old) — without any ETL pipeline or touching the Aurora production cluster.

■ Enable Aurora Zero-ETL to Redshift

1. Aurora: ensure engine version supports Zero-ETL (Aurora PostgreSQL 15.4+)
2. Redshift: create a Redshift Serverless workgroup (or provisioned cluster)
3. Aurora → Zero-ETL integrations → Create integration
4. Source: shopflow-aurora-cluster | Target: shopflow-redshift-namespace
5. Databases to replicate: shopflow (all tables) or specific tables
6. Click Create — replication starts within 5 minutes
7. In Redshift: new database appears: awsdatacatalog.shopflow.*
8. Run analytics queries directly in Redshift without any ETL pipeline

---

06-20 Aurora Maintenance Windows and Patching

Aurora patches the database engine automatically during a weekly maintenance window. Understanding and configuring this window prevents surprise downtime during business hours. ShopFlow sets the maintenance window to Sunday 2:00-3:00 AM UTC (Sunday night US Eastern = Monday morning — low traffic for a US-focused platform).

Maintenance Event	Impact	Duration	ShopFlow Strategy
Minor version patch (e.g., 15.4 → 15.5)	Brief failover: 20-30 seconds for writer to restart	20-30 seconds	Auto Minor Version Upgrade: enabled; happ
Major version upgrade (e.g., 15 → 16)	Significant downtime: 10-30 minutes; must be initiated manually	10-30 minutes	Test on staging first; schedule with maintena
OS patching	Brief failover on each instance; readers first, then writer	30-60 seconds per instance	Configure multi-AZ; readers patched first wh
Hardware replacement	AWS migrates instance to new hardware; brief disruption	Seconds (if using Aurora; longer for RDS standard)	Aurora handles this automatically; no action requ

■ Configure Maintenance Window

1. RDS → Databases → shopflow-aurora-cluster → Modify
2. Maintenance window: Select a window → Sun:02:00-Sun:03:00 (UTC)
3. Enable auto minor version upgrade: Yes (security patches applied automatically)
4. Apply modifications: Apply during the next scheduled maintenance window
5. Set up CloudWatch Events: rule on RDS maintenance events → SNS → email alert to ops team

06-21 Aurora Database Activity Streams — Compliance Auditing

Database Activity Streams captures every database query, connection, and data access event and streams it to Kinesis Data Streams in near real-time. Required for PCI DSS compliance: ShopFlow must log every access to the payment and customer PII tables. Activity Streams runs asynchronously (does not add latency to queries) and cannot be disabled by database users — only by AWS account administrators.

Activity Stream Feature	Description	ShopFlow Use Case
What is captured	Every SQL statement, user, object accessed, result count, timestamp	Audit: "Who SELECTed from the payments table in the last 24 h
Stream destination	Kinesis Data Stream (then Firehose → S3 for long-term storage)	Stream to S3 for 7-year PCI DSS audit log retention
Encryption	All events encrypted with KMS key	Use a separate KMS key for audit logs from the database encrypt
Performance impact	Asynchronous mode: near-zero latency impact	Enable asynchronous mode for production; synchronous mode a
Tamper resistance	Cannot be disabled by database superusers; requires AWS account admin	Satisfies PCI DSS Requirement 10.3: protect audit logs from mod

06-BP Best Practices and Anti-Patterns

✓ Always enable deletion protection on Aurora clusters — takes 2 seconds to enable, prevents catastrophic accidents; you must explicitly disable it before decommissioning

✓ Use the reader endpoint for all read queries — route SELECT queries to the Aurora reader endpoint; only writes go to the cluster writer endpoint. This distributes load and reduces write instance pressure

✓ Enable Performance Insights from day one — it is free for 7-day retention; it tells you which queries are slow before customers complain

✓ Use connection pooling (HikariCP) with appropriate pool sizes — too large and you hit Aurora max_connections; too small and requests queue under load. Formula: (2 × vCPU) + 1 = max connections per instance

✓ Store database credentials in Secrets Manager, not application.yml — enable automatic rotation; never hardcode the password anywhere

✓ Use Flyway or Liquibase for all schema changes — version-controlled SQL migrations ensure all environments are in sync and changes are auditable and reversible

■ Never run ALTER TABLE on large tables without testing migration time first — ALTER TABLE orders ADD COLUMN on a 500M row table can take hours and block all writes. Use online DDL tools (pg_repack) or Aurora Fast DDL feature for large tables

■ Do not use general_purpose storage class for Aurora — Aurora manages its own distributed storage; you choose instance type (compute), not storage type. For high IOPS needs, upgrade the instance type, not a storage setting

06-QR Quick Reference and Interview Q&A

Question	Answer
What is the difference between Aurora Serverless and Aurora Provisioned?	Aurora Provisioned is for specific instance sizes (db.r6g.large) that run continuousl
How does Aurora handle a writer instance failure?	Aurora continuously maintains two warm reader replicas. When the writer fails: (1) Aurora detects failu
What is Aurora Global Database?	A Global Database spans multiple AWS regions. One primary region has the writer; up to 5 secondary